Práctica Flex

Pablo Linari Pérez

Diciembre 2024

Contents

1 Introducción **3**

2 Definiciones **3**

3 Reglas **5**

4 Procedimientos **7**

4.1 Procesar tags 7
4.2 Procesar listas 8
4.3 Procesar títulos 9
4.4 Función principal 10

5 Ejecución **11**

1 Introducción

En esta práctica se usa la herramienta `flex++` para realizar un programa que pase archivos HTML a formato Markdown,con el objetivo de que sea más fácil de leer y que se pueda visualizar de manera correcta en aplicaciones que lo soporten como GitHub u Obsidian.

2 Definiciones

La primera parte del programa consiste en la definición de las expresiones que estaremos buscando para reemplazarlas por su correspondiente equivalente en Markdown. Debemos tener en cuenta que en Markdown la sintaxis es más reducida que en HTML, ya que no incluye opciones de estilo ni `divs`, entre otros. Usaremos las siguientes equivalencias:

- `h1` = #
- `h2` = ##
- `h3` = ###
- `p` = texto plano
- `ul` = lista desordenada
- `ol` = lista ordenada
- `head` = título de página

Listing 1: Definiciones

```
%option noyywrap
%{
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Variables para contar l neas , palabras y caracteres
ifstream infile;
void processtag(string c);
void processorderedlist(string c);
void processunorderedlist(string c);
void processtitle(string c);

%}

texto (.|\s)*
htmltag <html.*>
```

```
htmltagc <\/html>
etiq1 <h1>
etiq1c <\/h1>
etiq2 <h2>
etiq2c <\/h2>
etiq3 <h3>
etiq3c <\/h3>
plaintext <p>
plaintextc <\/p>
body <body>
bodyc <\/body>
ulist <ul>
ulistc <\/ul>
olist <ol>
olistc <\/ol>
main <main>
mainc <\/main>
header <head>
headerc <\/head>
any (\t|\n)*


tagh1 {etiq1}{texto}{etiq1c}
tagh2 {etiq2}{texto}{etiq2c}
tagh3 {etiq3}{texto}{etiq3c}
tagtext {plaintext}{texto}{plaintextc}
unorderedlist {ulist}({any}|{texto})*{ulistc}
orderedlist {olist}({any}|{texto})*{olistc}
titulo {header}({any}|{texto})*{headerc}
doctype <!DOCTYPE.*>
```

En esta parte del código se puede observar la declaración de las funciones necesarias para realizar los cambios y las asignaciones de las expresiones regulares. Las 20 asignaciones primeras tienen el mismo objetivo: identificar los diferentes tags que envuelven el texto en HTML. Las variables de tags de cierre tienen el mismo nombre que su tag de apertura, pero con una "c" al final. Entre las 20 primeras cabe destacar la variable `texto`, la cual usamos en otras declaraciones para indicar que entre dos expresiones pueden ir cualquier tipo de caracteres, y la variable `any`, la cual usamos para indicar que puede haber cualquier tipo de espacio, tanto de salto de línea como de tabulador. La mayoría de las veces la usaremos junto a la variable `texto`.

En la segunda parte de las declaraciones, podemos observar expresiones regulares que derivan de las declaradas previamente, las cuales usaremos en el siguiente apartado para clasificar el tipo de cada texto. Vemos que todas consisten en expresiones regulares formadas por las 20 primeras variables previamente declaradas. Estas expresiones tratan de identificar cada texto dentro de su correspondiente tag para que, en la parte de reglas, sea posible formatearlo con facilidad.

3 Reglas

En esta sección veremos los procedimientos que se ejecutan cada vez que detectamos una expresión que concuerda con una de nuestras expresiones regulares previamente descritas.

Listing 2: Reglas

```
{doctype}|({main}|{mainc})|({body}|{bodyc})
|({htmltag}|{htmltagc}) {cout << "";}
{titulo}                    {processtitle(yytext);}
{tagh1}                     {processtag(yytext);}
{tagh2}                     {processtag(yytext);}
{tagh3}                     {processtag(yytext);}
{tagtext}                   {processtag(yytext);}
{orderedlist}               {processorderedlist(yytext);}
{unorderedlist}             {processunorderedlist(yytext);}
```

En este programa, cada expresión regular se utiliza para identificar ciertas estructuras del archivo HTML y procesarlas adecuadamente. A continuación se describe el comportamiento de cada expresión regular y su función:

- **{doctype}**‖**{body}** ‖**{bodyc}**‖**{main}** ‖**{mainc}**‖**{htmltag}** ‖ **{htmltagc}**: Esta expresión regular coincide con las declaraciones de tipo de documento HTML, como `<!DOCTYPE ...>`,`<main>`,`<body>`,`<html>` y sus correspondientes etiquetas de cierre . Las cuales son expresiones que no tienen uso en el formato Markdown y por tanto se sustituyen por espacios en blanco .

- **{titulo}**: Esta expresión regular coincide con la etiqueta `<head>` y su correspondiente etiqueta de cierre `</head>`. Cuando se encuentra, se llama a la función `processtitle(yytext)`, que procesa el contenido dentro de estas etiquetas, eliminando las etiquetas `<h*>` y `</h*>` y agregando un formato específico al título extraído.

- **{tagh1}**: Coincide con la etiqueta de apertura `<h1>` y su correspondiente etiqueta de cierre `</h1>`. Cuando se detecta, se llama a la función `processtag(yytext)`, que reemplaza la etiqueta `<h1>` por el símbolo de Markdown `#` y el contenido dentro de estas etiquetas se mantiene para ser procesado y formateado.

- **{tagh2}**: Similar a {tagh1}, esta expresión regular coincide con las etiquetas `<h2>` y `</h2>`. Cuando se detecta, se llama a la función `processtag(yytext)`, que reemplaza la etiqueta `<h2>` por `##` en formato Markdown.

- **{tagh3}**: Coincide con las etiquetas `<h3>` y `</h3>`. Similar a las anteriores, se llama a la función `processtag(yytext)`, que reemplaza la etiqueta `<h3>` por `###` en formato Markdown.

- **{tagtext}**: Coincide con las etiquetas `<p>` y `</p>`, que definen párrafos en HTML. Se llama a la función `processtag(yytext)` para procesar el

contenido de los párrafos, aunque en este caso no se realiza ninguna modificación en el formato, ya que simplemente se imprime el texto sin las etiquetas.

- **{orderedlist}**: Coincide con la etiqueta de apertura `<ol>` y su cierre `</ol>`, lo que indica una lista ordenada en HTML. Cuando se detecta, se llama a la función `processorderedlist(yytext)`, que convierte las etiquetas de la lista ordenada en un formato adecuado para Markdown, añadiendo numeración a los elementos de la lista.

- **{unorderedlist}**: Coincide con la etiqueta de apertura `<ul>` y su cierre `</ul>`, lo que indica una lista desordenada en HTML. Cuando se detecta, se llama a la función `processunorderedlist(yytext)`, que convierte las etiquetas de la lista desordenada en formato Markdown, añadiendo un guion antes de cada elemento de la lista.

4 Procedimientos

4.1 Procesar tags

Listing 3: Procesar tags

```
void processtag(string c) {
        string tag = c.substr(0,4);
        string replace ="";
        string cad = c.substr(4,c.size()-9);
        if(tag == "<h1>"){
            replace ="#␣";
        }else if(tag =="<h2>"){
            replace = "##␣";
        }else if(tag == "<h3>"){
            replace ="###␣";
        }else {
            cad = c.substr(3,c.size()-7);
            replace ="";
        }
        cout << replace << cad<<endl;
}
```

Esta función procesa etiquetas de encabezado HTML (`<h1>`, `<h2>`, `<h3>`) y las convierte a su equivalente en formato Markdown (#, ##, ###). El texto contenido entre las etiquetas se extrae y se imprime junto al prefijo correspondiente. Si no se trata de un encabezado, elimina las etiquetas HTML sin añadir prefijos.Este caso se considera para las etiquetas de la forma `<p>` las cuales en Markdown no llevan prefijo .

4.2 Procesar listas

Listing 4: Procesar listas no ordenadas

```
void processunorderedlist(string c) {
    int pos = c.find("<ul>");
    string cad =c;
    cad.erase(pos,4);
    pos = cad.find("</ul>");
    cad.erase(pos,5);
    while (true){
        pos = cad.find("<li>");
        if(pos == string::npos) break;
        cad.erase(pos,4);
        cad = cad.insert(pos,"\n␣-");
        pos =cad.find("</li>");
        cad.erase(pos,5);
    }

    cout << cad << endl;
}
```

Esta función procesa listas desordenadas definidas por las etiquetas `<ul>` y `<li>`. Elimina las etiquetas HTML `<ul>` y `</ul>` y convierte cada elemento de la lista (`<li> ... </li>`) en formato Markdown, precedido por un guion (`-`) para marcar los elementos de la lista.Este proceso se repite en un bucle infinito hasta que no quedan elementos dentro de la sección de `<ul>`.

Listing 5: Procesar listas ordenadas

```
void processorderedlist(string c) {
    int pos = c.find("<ol>");

    string cad =c;
    string aux="";
    cad.erase(pos,4);
    pos = cad.find("</ol>");
    cad.erase(pos,5);
    int cont =0;
    while (true){
        pos = cad.find("<li>");
        cont++;
        if(pos == string::npos) break;
        cad.erase(pos,4);
        aux+="\n"+to_string(cont)+".␣";
        cad = cad.insert(pos,aux);
        aux="";
        pos =cad.find("</li>");
        cad.erase(pos,5);
    }

```

```
    cout << cad << endl;
}
```

Esta función procesa listas ordenadas definidas por las etiquetas `<ol>` y `<li>`. Elimina las etiquetas HTML `<ol>` y `</ol>` . Su funcionamiento es parecido al de la función anterior, pero esta vez, en vez de reemplazar las etiquetas `<li>` por `-` las reemplaza por el respectivo número que les tocaría según su orden de aparición en la lista.

4.3 Procesar títulos

Listing 6: Procesar títulos

```
void processtitle(string c){
    int pos = c.find("<head>");
    c.erase(pos,8);
    pos =c.find("</head>");
    c.erase(pos,9);
    pos = c.find("<h");
    c.erase(pos,4);
    pos = c.find("</h");
    c.erase(pos,5);

    c+="\n-----------------------------------------\n";
    c="\n-----------------------------------------\n"+c;
    cout << c;
}
```

Esta función procesa los títulos definidos en la sección `<head>` del HTML. Elimina las etiquetas `<head>`, `</head>`, y las que son de la forma `<h*>`, y `</h*>` que son usadas para darle formato al header . El texto del título se enmarca con líneas de separación antes y después, usando caracteres de guion (`-`) para resaltarlo.

4.4 Función principal

Listing 7: main

```
int main(int argc, char **argv) {
    if (argc != 2) {
        cerr << "Usage: " << argv[0] << " <input_html_file>"
            << endl;
        return 1;
    }

    infile.open(argv[1]);
    if (!infile.is_open()) {
        cerr << "Failed to open file: " << argv[1] << endl;
        return 1;
    }
        yyFlexLexer flujo (&infile,0);
        flujo.yylex();
    infile.close();


    return 0;
}
```

La función principal maneja la entrada del programa. Verifica que se pase exactamente un archivo como argumento. Luego, abre el archivo HTML especificado y lo procesa usando `Flex`. Finalmente, cierra el archivo y termina la ejecución.

5 Ejecución

Veamos ahora un ejemplo de uso del programa sobre un archivo HTML común el cual usa distintos tipos de listas y tipados para títulos y textos . La información no relevante para Markdown, como las etiquetas `body` o `html` directamente son borradas.

Listing 8: index.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <h1>Ejemplo  de HTML</h1>
</head>
<body>

    <h1>Este es un ejemplo de un archivo html </h1>
    <p>vamos a ver el funcionamiento de este programa  el
        cual</p>
    <p>se encarga de pasar un archivo html a markdown</p>

    <h1>Este es un texto h1</h1>
    <p>En markdown el texto h1 se indica con #</p>
    <h2>Este es un texto h2</h2>
    <p>En markdown el texto h2 se indica con ##</p>
    <h3>Este es un texto h3</h3>
    <p>En markdown el texto h3 se indica con ###</p>

    <h2>Listas desordenadas</h2>
    <p>las listas desordenadas se ven con - en markdown</p>
    <ul>
        <li>Primer elemento desordenado </li>
        <li>Segundo elemento desordenado </li>
        <li> Tercero elemento desordenado </li>
        <li> Cuarto elemento desordenado </li>
    </ul>
    <h2>Listas ordenadas</h2>
    <p>las listas ordenadas se ven con 1. en markdown</p>
    <ol>
        <li>Primer elemento ordenado </li>
        <li>Segundo elemento ordenado </li>
        <li> Tercero elemento ordenado </li>
        <li> Cuarto elemento ordenado </li>
    </ol>
</body>
</html>
```

El archivo resultado tras ejecutar el programa con la siguiente sentencia `./htmltomd index.html > salida.md` es :

Listing 9: salida.md

```
-------------------------------------------------------------

   Ejemplo  de HTML

-------------------------------------------------------------



    # Este es un ejemplo de un archivo html

    vamos a ver el funcionamiento de este programa  el cual

    se encarga de pasar un archivo html a markdown


    # Este es un texto h1

    En markdown el texto h1 se indica con #

    ## Este es un texto h2

    En markdown el texto h2 se indica con ##

    ### Este es un texto h3

    En markdown el texto h3 se indica con ###


    ## Listas desordenadas

    las listas desordenadas se ven con - en markdown



 -Primer elemento desordenado

 -Segundo elemento desordenado

 - Tercero elemento desordenado

 - Cuarto elemento desordenado


    ## Listas ordenadas

    las listas ordenadas se ven con 1. en markdown

```

```


1. Primer elemento ordenado

2. Segundo elemento ordenado

3.  Tercero elemento ordenado

4.  Cuarto elemento ordenado
```